

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 08

NOMBRE COMPLETO: Tapia Verastegui Venancio Miguel

Nº de Cuenta: 320324187

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 17/10/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios.

Iluminación 2 en una nave espacial de dos motores

Se importó una nave con dos reactores y se añadió flotando a la escena de E08. Se añadieron una luz puntual y una spotlight para la nave, y una puntual azul con plasma texturizado por motor. Las teclas controlan las fuentes y los efectos de forma sincronizada.

Ejercicio 1 Importación y luces de la nave

Se utilizó craft_speederB de Space Kit de Kenney, con licencia CC0. Se conservaron vértices y normales y se adaptaron UV y materiales para cuatro texturas de paneles. ASSIMP carga nave.obj y nave.mtl. Se conservan el piso, skybox, avión, dado, Holocron y Rover de E08. La nave se añade en (0, 4, -10), con escala 3.5 y giro de 180 grados, flotando sobre el piso original en Y = -2.

```
Scene.cpp
 25  ship.LoadModel("Models/nave.obj");
 79  shipMatrix=glm::scale(glm::rotate(glm::translate(glm::mat4(1),
     shipPosition), glm::radians(180.f), glm::vec3(0, 1, 0)),
     glm::vec3(3.5f));
Scene.h
 27  PointLight shipPoint{1, .72f, .30f, 0, 3.5f, 0, 0, 0, 1, .09f, .035f};
 29  SpotLight shipSpot{1, .85f, .55f, 0, 6, 0, 0, 0, 0, -1, 0, 1, .08f,
     .025f, 28};
```

Figura 1. Importación del OBJ, transformación y definición de las dos fuentes de la nave.

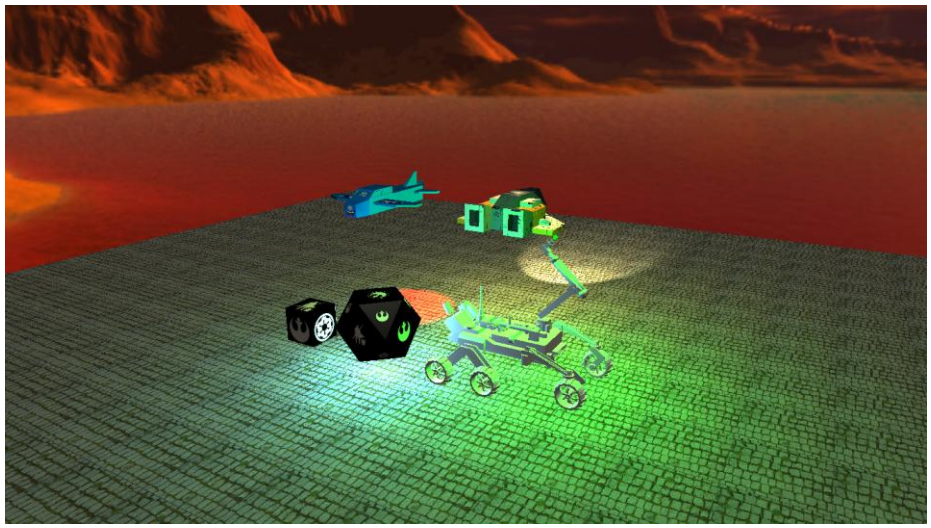


Figura 2. Nave flotando, texturizada y con sus luces encendidas; ambos motores están apagados.

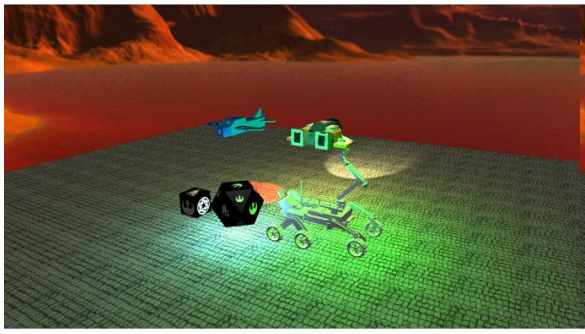
Ejercicio 2 Encendido conjunto con una tecla

La tecla L invierte una sola bandera, shipLights, que controla al mismo tiempo la luz puntual y la spotlight de la nave. Cuando está apagada, ninguna de estas dos fuentes se envía al shader. Los motores utilizan banderas diferentes, de modo que L no cambia su estado.

```
Window.cpp
 27  if(action!=GLFW_PRESS)return;
 33  case GLFW_KEY_L: shipLights=!shipLights; break;
Scene.cpp
 39  shipPoint.SetPosition( lightPosition);
 42  shipSpot.SetFlash(spotPosition, spotDirection);
 44  if(window.ShipLightsOn()){activePoints[pointCount++]=shipPoint;
     activeSpots[spotCount++]=shipSpot; }
```

Figura 3. Conmutación en Window.cpp y uso de una bandera común para las dos luces.

L: encendidas / motores OFF



L: apagadas / motores OFF

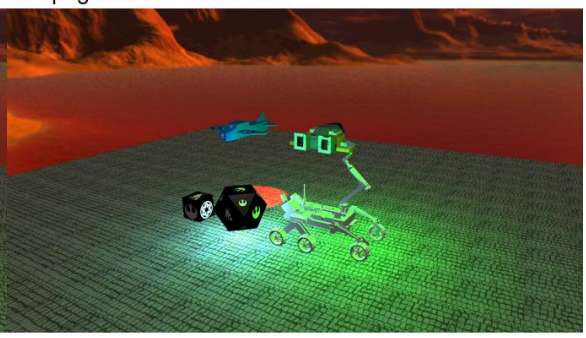


Figura 4. Comparación con las dos luces de la nave encendidas y apagadas, sin motores activos.

Ejercicio 3 Plasma azul y luces de los reactores

Se creó una textura PNG RGBA con núcleo claro, borde azul y transparencia gradual. Cada chorro utiliza dos planos cruzados situados en su tobera. El efecto se dibuja después del modelo, con mezcla aditiva GL_SRC_ALPHA / GL_ONE y sin escribir profundidad mientras se dibuja el plasma. La prueba de profundidad permanece activa para que el casco oculte las partes que correspondan.

```
Scene.cpp
 63  glUniform1i(plasma, 1); flameTexture.UseTexture(); glEnable(GL_BLEND);
     glBlendFunc(GL_SRC_ALPHA, GL_ONE); glDepthMask(GL_FALSE);
 66  if(window.EngineOn(index)){
 67  model=glm::rotate(glm::translate(glm::mat4(1), enginePosition[index]),
     glm::radians(180.f), glm::vec3(0, 1, 0));
 68  glUniformMatrix4fv(shader.GetModelLocation(), 1, GL_FALSE,
     glm::value_ptr(model)); flame.RenderMeshModel();
 71  glDepthMask(GL_TRUE); glDisable(GL_BLEND); glUniform1i(plasma, 0);
```

Figura 5. Dibujo del plasma con transparencia y restauración de los estados de OpenGL.

En cada motor se coloca una luz puntual azul junto al inicio del chorro. La textura simula la emisión y la luz ilumina las superficies cercanas; son dos elementos distintos que comparten el mismo estado. El brillo del plasma varía suavemente con el tiempo.

Ejercicio 4 Control independiente de ambos motores

La tecla 1 alterna el motor 1 y la tecla 2 alterna el motor 2. La misma bandera de cada motor decide si se añade su luz puntual azul y si se dibuja su textura de plasma. Se aceptan también las teclas 1 y 2 del teclado numérico. Solo GLFW_PRESS produce la conmutación, por lo que GLFW_REPEAT no provoca cambios adicionales al sostener una tecla.

```
Window.cpp
 34  case GLFW_KEY_1: case GLFW_KEY_KP_1: engines[0]=!engines[0]; break;
 35  case GLFW_KEY_2: case GLFW_KEY_KP_2: engines[1]=!engines[1]; break;
Scene.cpp
 46  enginePosition[index]=glm::vec3(shipMatrix*glm::vec4(nozzles[index],
     1));
 47  enginePoints[index].SetPosition(
     enginePosition[index]+glm::vec3(shipMatrix*glm::vec4(0, 0, -.16f, 0)));
 48  if(window.EngineOn(index))activePoints[pointCount++]=enginePoints[index];
```

Figura 6. Teclas independientes y actualización de las luces desde las toberas de la nave.

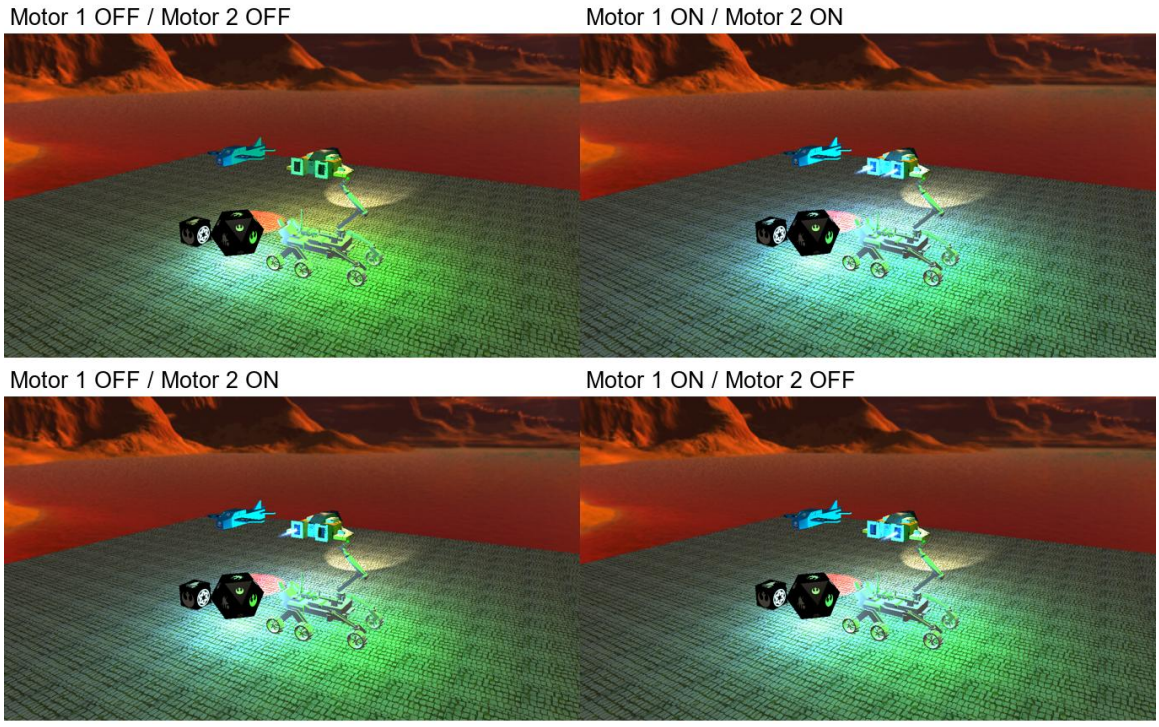


Figura 7. Los cuatro estados requeridos de los motores en la misma vista.

Se verificaron los estados 00, 11, 01 y 10. En cada motor apagado desaparecen simultáneamente su chorro y su luz azul, mientras el otro conserva su estado. Se amplió el límite del shader a cinco luces puntuales: las dos de E08, una de la nave y dos de los motores. La capacidad de cuatro spotlights permite conservar también el faro y la linterna anteriores.

Organización del teclado y del ciclo principal

El manejo de eventos, las teclas y las banderas están declarados en Window.h e implementados en Window.cpp. Scene conserva como miembros el tiempo, las matrices, las luces y los arreglos utilizados durante el dibujo. Los recursos se cargan en la inicialización; el while del main únicamente llama a Frame y no declara variables ni construye luces.

```
P08-320324187.cpp
  1  // Practica 08 Iluminacion 2 - 320324187
  2  #include "Scene.h"
  3  #include <string>
  4  int main(int argc, char** argv){
  5  Scene scene;
  6  const bool evidence=argc>1&&std::string(argv[1])=="--evidencia";
  7  if(!scene.Initialise(argv[0], evidence))return 1;
  8  // Sin declaraciones de variables, lectura de teclado o construccion de
     luces.
  9  while(!scene.ShouldClose()){
 10  scene.Frame();
 11  }
 12  return scene.Shutdown();
 13  }
```

Figura 8. Main separado y ciclo principal sin manejo directo del teclado.

```
Window.h
 29  bool keys[1024]{};
 30  bool shipLights=true, engines[2]{false, false}, orbit=false;
Scene.h
 30  PointLight activePoints[5];
 32  unsigned int pointCount=0, spotCount=0;
 33  glm::mat4 projection{1}, view{1}, shipMatrix{1}, model{1};
 37  double last=0, now=0, sceneTime=0;
 38  float dt=0;
```

Figura 9. Estados y variables persistentes definidos en Window y Scene.

Controles: L prende o apaga las dos luces de la nave; 1 y 2 alternan los motores. WASD desplaza la cámara, Q/E baja y sube la cámara y el mouse cambia la vista. I/K cambia la altura del avión, R activa la órbita y Esc cierra el programa. C cambia el color del faro, H y P alternan las luces del Holocron y de la pinza; 7/8 mueven el brazo y las flechas desplazan el Rover, como en E08.

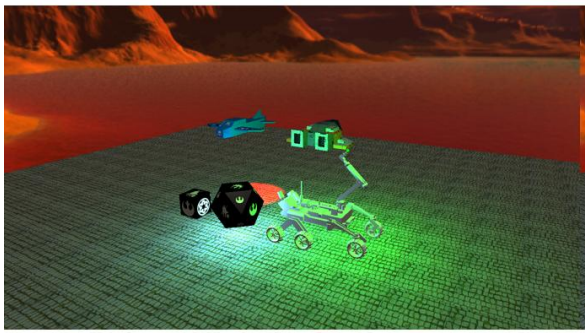
Comprobación de ejecución

Se compiló en Release x64 y se ejecutó el modo --evidencia. La prueba del manejador de teclado comprobó pulsación, repetición e independencia. Después se guardaron ocho escenas: cuatro combinaciones de motores con L encendida y las mismas cuatro con L apagada. En las ocho se obtuvo error de OpenGL igual a 0.

```
verificacion.txt
  1  Prueba callback Window: L/1/2, repeticion e independencia OK.
  2  nave-on-motores-00 puntos=3 spots=2 GL=0
  3  nave-on-motores-10 puntos=4 spots=2 GL=0
  4  nave-on-motores-01 puntos=4 spots=2 GL=0
  5  nave-on-motores-11 puntos=5 spots=2 GL=0
  6  nave-off-motores-00 puntos=2 spots=1 GL=0
  7  nave-off-motores-10 puntos=3 spots=1 GL=0
  8  nave-off-motores-01 puntos=3 spots=1 GL=0
  9  nave-off-motores-11 puntos=4 spots=1 GL=0
```

Figura 10. Registro real de las ocho combinaciones, conteo de fuentes y resultado de OpenGL.

Luces nave OFF / motores OFF



Luces nave OFF / motores ON

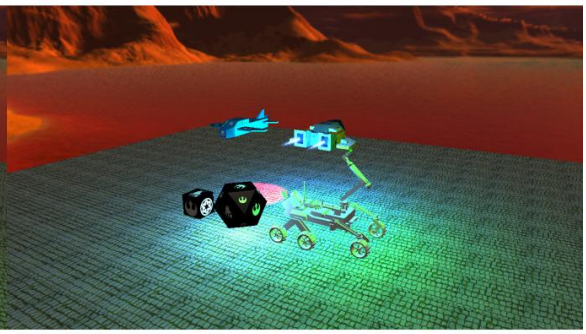


Figura 11. Los motores pueden funcionar incluso cuando las dos luces de la nave están apagadas.

Con las luces de la nave encendidas y las de E08 activas hay tres puntuales y dos spotlights. Los motores añaden una o dos puntuales azules. Al apagar L permanecen las dos puntuales de E08 y el faro del avión; solamente se retiran las dos fuentes de la nave. Se mantiene la luz direccional de E08.

2.- Problemas presentados y soluciones.

Al separar la escena en archivos, stb_image se incluyó inicialmente más de una vez con la implementación habilitada y el compilador reportó definiciones duplicadas. Se corrigió dejando STB_IMAGE_IMPLEMENTATION únicamente en Texture.cpp. La versión final volvió a compilar correctamente.

Los primeros chorros se veían demasiado delgados. Se ajustó la distribución del canal alfa y el ancho de la textura para distinguir el plasma azul. También se dibujan al final con mezcla aditiva y se restaura glDepthMask al terminar, para no afectar al resto de la escena.

Se revisaron las coordenadas de las toberas del modelo importado para que el plasma no apareciera separado del motor. Tanto las luces como los planos del efecto utilizan las posiciones calculadas desde shipMatrix. La flotación mantiene esa unión. La integración conserva las transformaciones, materiales, texturas y controles de los objetos anteriores de E08.

La captura de las ocho escenas no registró errores de OpenGL. Las imágenes y el registro de verificación corresponden a la versión final. El proyecto conserva advertencias de conversión de tipos en clases de la base; no impidieron la compilación ni la ejecución comprobada.

3.- Conclusión:

a. Complejidad y explicación de los ejercicios.

La parte más compleja fue sincronizar el efecto de plasma con la luz puntual de cada motor y mantener independencia entre las tres banderas. Compartir una bandera por motor evita estados inconsistentes, como un chorro visible sin su iluminación. Transformar las posiciones locales con la matriz de la nave mantiene unidos los emisores al modelo.

b. Comentarios generales.

Resultó útil comparar las fuentes por separado y revisar el conteo que recibe el shader. Para explicar este proceso conviene mostrar primero la posición local de una tobera y después su transformación al espacio mundial, junto con la bandera que controla su dibujo.

c. Conclusión.

Se integró una nave importada y texturizada con dos motores controlables. Las teclas permiten apagar y encender en conjunto las dos luces de la nave y controlar por separado los dos chorros y sus luces azules. La organización en Window y Scene mantiene el ciclo principal breve y permite reutilizar los recursos durante la ejecución.

1. Bibliografía en formato APA.

GLFW. (s. f.). Input guide. Recuperado el 17 de octubre de 2026, de https://www.glfw.org/docs/latest/input_guide.html

Kenney. (2020). Space Kit [Modelos tridimensionales, licencia CC0]. https://kenney.nl/assets/space-kit

Recursos utilizados

Modelo: craft_speederB del paquete Space Kit. Se conservaron su geometría y normales, y se adaptaron UV y texturas de paneles para esta escena. La textura de plasma azul y las imágenes de paneles se prepararon para el proyecto. El suelo y las seis caras del skybox se conservaron de E08 sin modificar sus archivos. La licencia y el enlace del modelo se conservan junto a los recursos entregados.

Archivos de entrega

P08-320324187.pdf contiene el reporte y P08-320324187.cpp es el main. Ambos se entregan separados de P08-320324187.zip. El ZIP contiene las clases, los shaders, los modelos, las texturas, las bibliotecas y el ejecutable. Para recompilar, el main se coloca junto al ZIP antes de extraerlo; la carpeta del proyecto queda a su lado y su ruta al main es relativa.